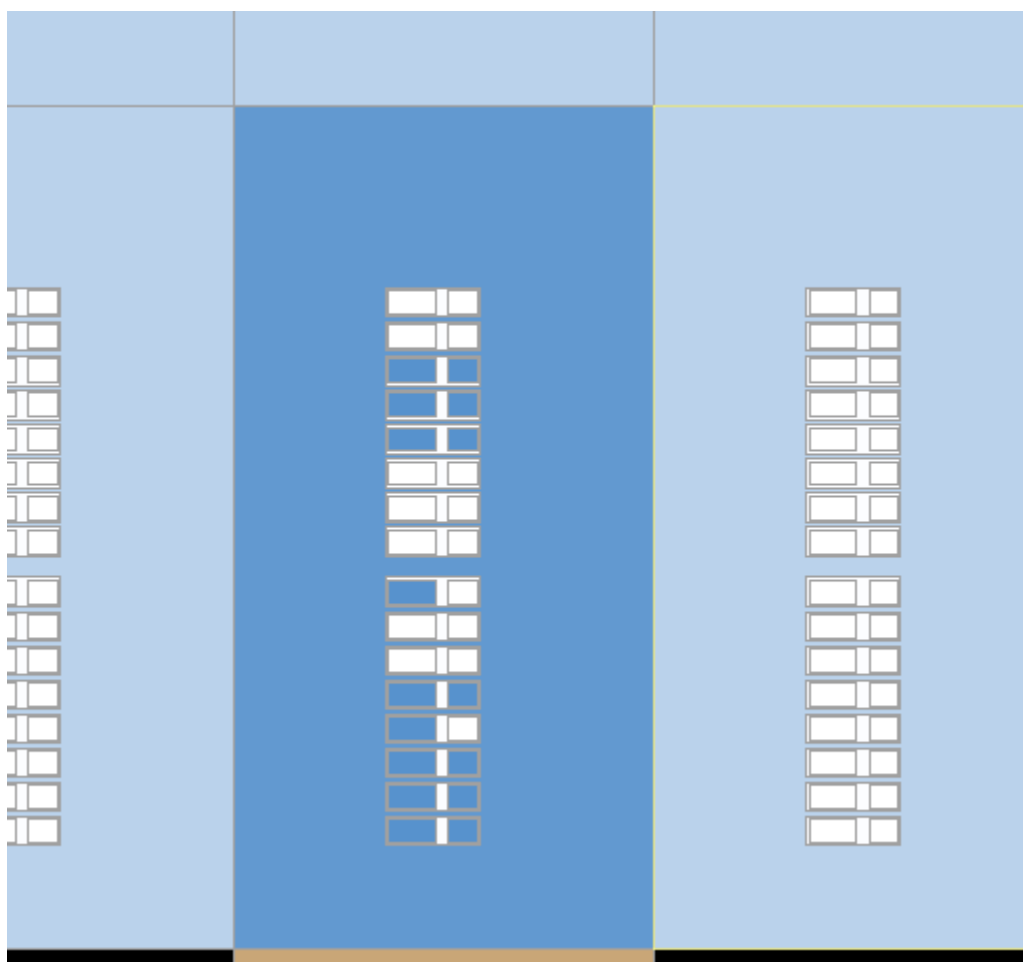
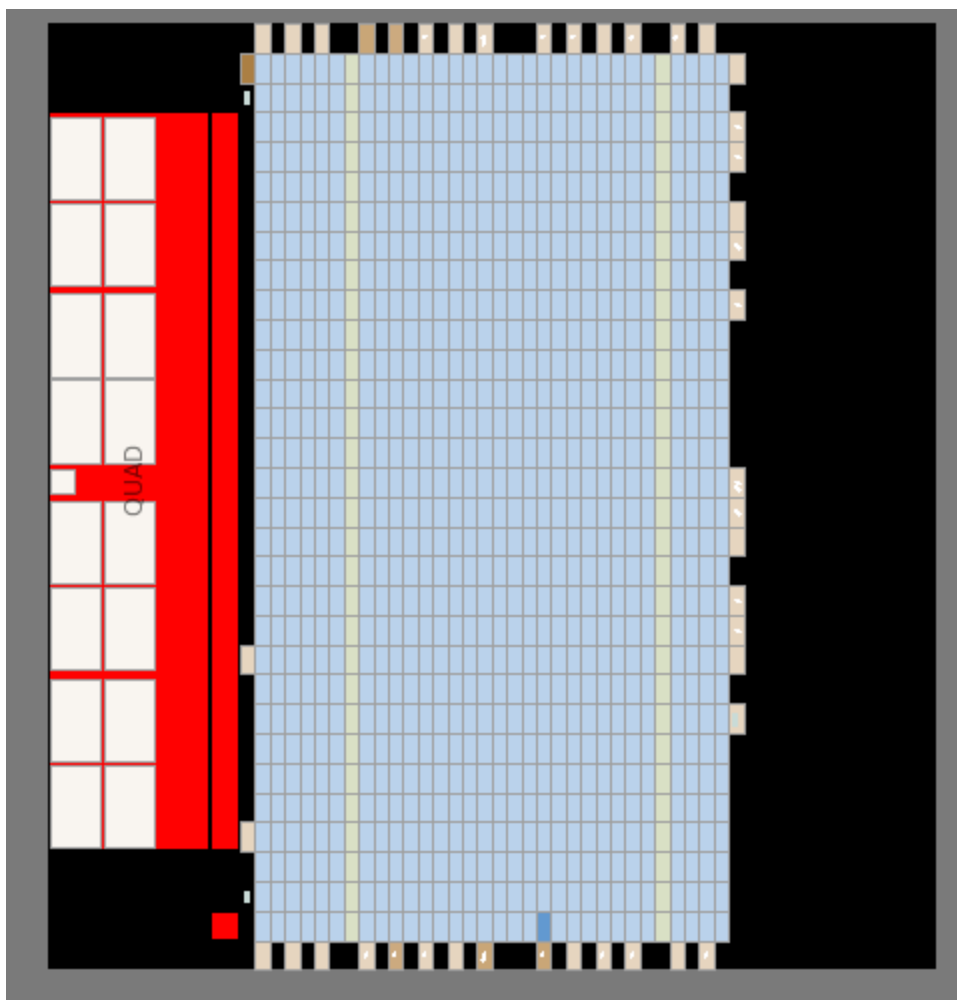


به نام خدا
مرتضی محمدی مطلق
810102511

عکس چپ ها





عکس تایمینگ

CA4_module.sv

Compilation Report - CA4_module

Table of Contents

> Analysis & Synthesis

> Fitter

> Assembler

> Timing Analyzer

- Summary
- Parallel Compilation
- Clocks

> Slow 1200mV 125C Model

> Slow 1200mV -40C Model

> Fast 1200mV -40C Model

- Setup Summary
- Hold Summary
- Recovery Summary
- Removal Summary
- Minimum Pulse Width Summ

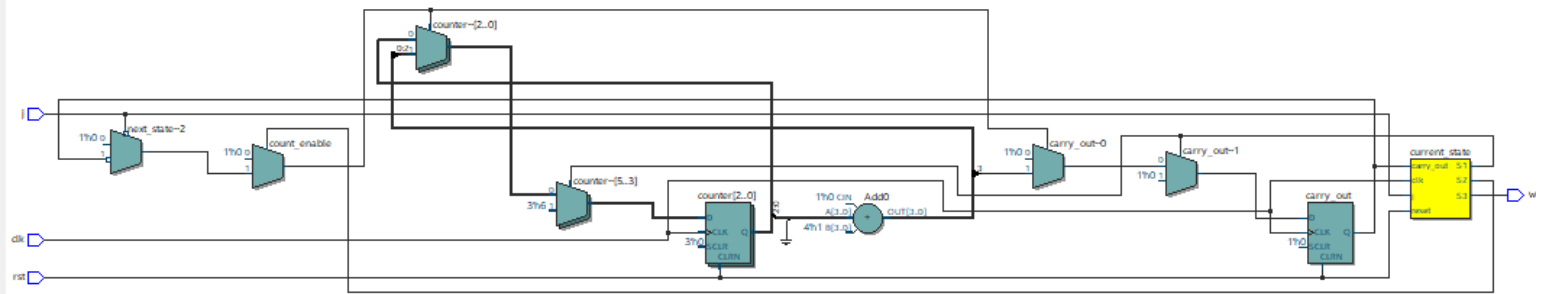
> Worst-Case Timing Paths

Fast 1200mV -40C Model Setup Summary

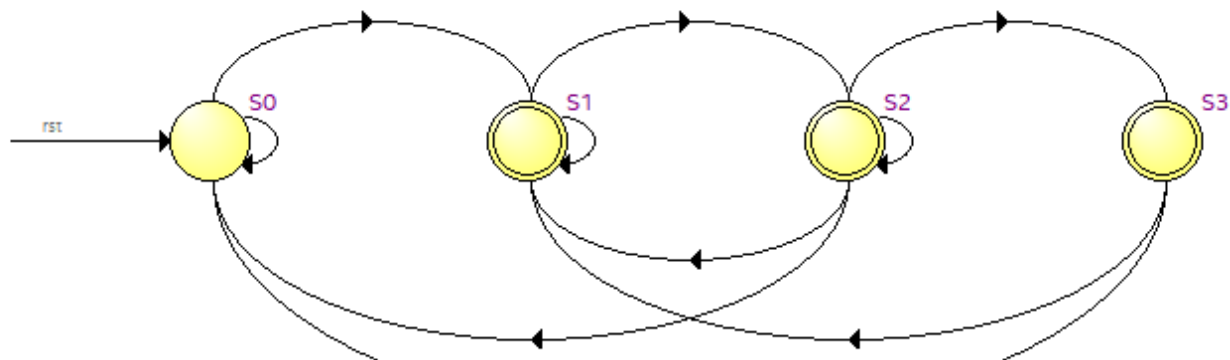
<<Filter>>

	Clock	Slack	End Point TNS
1	clk	0.047	0.000

عکس RTL

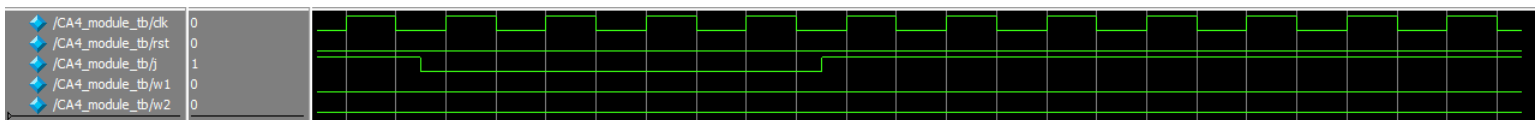


عکس های استیت ماشین



State Table			
	Source State	Destination State	Condition
1	S0	S0	(!j)
2	S0	S1	(j)
3	S1	S1	(!j)
4	S1	S2	(j)
5	S2	S0	(!j).(carry_out)
6	S2	S1	(j).(!carry_out)

شکل موج



تست بنچ کد های وریلاگ

Ln#	
1	`timescale 1ns/1ns
2	
3	module CA4_module_tb();
4	
5	logic clk, rst, j;
6	logic w1, w2;
7	
8	CA4_modulep uut2 (
9	.clk(clk),
10	.rst(rst),
11	.j(j),
12	.w(w2)
13	);
14	
15	CA4_module uut1 (
16	.clk(clk),
17	.rst(rst),
18	.j(j),
19	.w(w1)
20	);
21	
22	initial begin
23	clk = 0;
24	forever #10 clk = ~clk;
25	end
26	
27	initial begin
28	rst = 1;
29	j = 0;
30	#25;
31	
32	rst = 0;
33	#20;
34	
35	send(1);
36	send(0);
37	repeat(3) send(0);
38	send(1);
39	send(1);

```

28     rst = 1;
29     j = 0;
30     #25;
31
32     rst = 0;
33     #20;
34
35     send(1);
36     send(0);
37     repeat(3) send(0);
38     send(1);
39     send(1);
40     send(0);
41
42     send(1);
43     send(0);
44     send(0);
45     send(1);
46
47     #100;
48     $finish;
49 end
50
51 task send(input logic bitval);
52 begin
53     j = bitval;
54     #40;
55 end
56 endtask
57
58 endmodule
59

```

کد های وریلاگ

Ln#	
1	<code>`timescale 1ns/1ns</code>
2	<code>module CA4_module (input clk, rst, j, output logic w);</code>
3	
4	<code>parameter [1:0] S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;</code>
5	
6	<code>logic [1:0] current_state, next_state;</code>
7	<code>logic [2:0] counter;</code>
8	<code>logic count_enable, load_counter;</code>
9	<code>logic carry_out;</code>
10	
11	<code>always @(posedge clk, posedge rst) begin</code>
12	<code>if (rst)</code>
13	<code>current_state <= S0;</code>
14	<code>else</code>
15	<code>current_state <= next_state;</code>
16	<code>end</code>
17	
18	<code>always @(current_state, j, counter, carry_out) begin</code>
19	<code>next_state = current_state;</code>
20	<code>load_counter = 0;</code>
21	<code>count_enable = 0;</code>
22	<code>w = 0;</code>
23	
24	<code>case (current_state)</code>
25	<code>S0: begin</code>
26	<code>if (j) next_state = S1;</code>
27	<code>else next_state = S0;</code>
28	<code>end</code>
29	
30	<code>S1: begin</code>
31	<code>load_counter = 1;</code>
32	<code>if (~j) next_state = S2;</code>
33	<code>else next_state = S1;</code>
34	<code>end</code>
35	
36	<code>S2: begin</code>
37	<code>if (~j) begin</code>
38	<code>if (~carry_out) begin</code>
39	<code>count_enable = 1;</code>

```

38         if (~carry_out) begin
39             count_enable = 1;
40             next_state = S2;
41         end else begin
42             next_state = S0;
43         end
44     end else begin
45         if (carry_out)
46             next_state = S3;
47         else
48             next_state = S1;
49         end
50     end
51
52     S3: begin
53         w = 1'b1;
54         if (j) next_state = S1;
55         else next_state = S0;
56     end
57 endcase
58 end
59
60 always @(posedge clk, posedge rst) begin
61     if (rst) begin
62         counter <= 0;
63         carry_out <= 0;
64     end
65     else if (load_counter) begin
66         counter <= 3'b011;
67         carry_out <= 0;
68     end
69     else if (count_enable) begin
70         {carry_out, counter} <= counter + 1;
71     end
72     else carry_out <= 0;
73 end
74 endmodule
75

```